

Subsetting & Vectorization

Lecture 04

Dr. Colin Rundel

Vectorization in R

Vectorized operations in R

Vectorized code applies one operation to many values without writing an explicit loop. R's arithmetic, comparison, and element-wise logical operators are vectorized.

```
1 x = 1:5  
2 x^2
```

R

```
[1] 1 4 9 16 25
```

```
1 sqrt(x)
```

R

```
[1] 1.000000 1.414214 1.732051 2.000000 2.
```

```
1 x + 10
```

R

```
[1] 11 12 13 14 15
```

```
1 x > 3
```

R

```
[1] FALSE FALSE FALSE TRUE TRUE
```

```
1 x %% 2 == 0
```

R

```
[1] FALSE TRUE FALSE TRUE FALSE
```

```
1 (x > 1) & (x < 5)
```

R

```
[1] FALSE TRUE TRUE TRUE FALSE
```

R recycles by length

When vector lengths differ, R repeats the shorter vector to match the longer one.

```
1 x = 1:6
```

R

```
1 x + 10
```

R

```
[1] 11 12 13 14 15 16
```

```
1 x + c(10, 100)
```

R

```
[1] 11 102 13 104 15 106
```

```
1 x + c(10, 100, 1000)
```

R

```
[1] 11 102 1003 14 105 1006
```

```
1 x + c(10, 100, 1000, 10000)
```

R

Warning in `x + c(10, 100, 1000, 10000)`: longer object length is a multiple of shorter object length

```
[1] 11 102 1003 10004 15 106
```

```
1 x > 3
```

R

```
[1] FALSE FALSE FALSE TRUE TRUE TRUE
```

```
1 x > c(2, 4)
```

R

```
[1] FALSE FALSE TRUE FALSE TRUE TRUE
```

```
1 x == c(1, 2, 3)
```

R

```
[1] TRUE TRUE TRUE FALSE FALSE FALSE
```

```
1 c(TRUE, FALSE) & c(TRUE, TRUE, TRUE)
```

R

Warning in `c(TRUE, FALSE) & c(TRUE, TRUE, TRUE)`: multiple of shorter object length

```
[1] TRUE FALSE TRUE
```

Zero-length recycling

Zero-length vectors contain no values; `NULL` also has length zero. For vectorized operations the presence of a length 0 vector means the result will also have length 0.

```
1 numeric()
numeric(0)
1 integer()
integer(0)
1 logical()
logical(0)
1 character()
character(0)
1 NULL
NULL
1 length(NULL)
[1] 0
```

```
1 x = 1:4
1 x + numeric()
numeric(0)
1 x > numeric()
logical(0)
1 c(TRUE, FALSE) & logical()
logical(0)
1 x + NULL
integer(0)
1 x == NULL
logical(0)
```

Vectorization in Python

Python lists are not vectorized

Python's list operators manipulate the container, not the values. Element-wise arithmetic and comparisons instead require explicit iteration: a `for` loop or, more idiomatically, a comprehension.

```
1 x = [1, 2, 3]
```

py

```
1 x * 2
```

py

```
[1, 2, 3, 1, 2, 3]
```

```
1 x + x
```

py

```
[1, 2, 3, 1, 2, 3]
```

```
1 x + 2
```

py

```
TypeError: can only concatenate list (not
```

```
1 x > 2
```

py

```
TypeError: '>' not supported between insta
```

```
1 x + [2]
```

py

```
[1, 2, 3, 2]
```

```
1 x > [2]
```

py

```
False
```

List comprehensions

A comprehension combines an expression with a `for` clause to construct a list. A trailing `if` filters values; a conditional expression transforms every value.

```
1 result = []
2 for x in range(6):
3     result.append(x**2)
4 result
```

py

[0, 1, 4, 9, 16, 25]

```
1 [x**2 for x in range(6)]
```

py

[0, 1, 4, 9, 16, 25]

```
1 [x**2 for x in range(10)
2  if x % 2 == 0]
```

py

[0, 4, 16, 36, 64]

```
1 [x if x % 2 == 0 else -1
2  for x in range(10)]
```

py

[0, -1, 2, -1, 4, -1, 6, -1, 8, -1]

Comprehensions can be nested. Additional `for` clauses act like nested loops and produce one flat list, while nested comprehensions build a list of lists.

```
1 [(i, j)
2  for i in range(3)
3  for j in range(2)]
```

py

[(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]

```
1 [[i * j for j in range(4)]
2  for i in range(3)]
```

py

[[0, 0, 0, 0], [0, 1, 2, 3], [0, 2, 4, 6]]

NumPy

NumPy is a third-party Python package for numerical computing. Its main data structure is the homogeneous, multidimensional `ndarray`, with operations designed to work efficiently across entire arrays (i.e. vectorization).

Installation happens once per project (e.g. `uv add numpy`). Before using NumPy in a Python session or script, import it; `np` is the conventional short alias, so functions are called as `np.array()`, `np.arange()`, `np.sqrt()`, etc.

```
1 import numpy as np
2 np.__version__
```

py

'2.5.2'

Vectorized operations with NumPy

NumPy arrays support the same style of vectorized arithmetic, comparison, and logical operations as R's atomic vectors, with similar performance compared to explicit for loops.

```
1 x = np.arange(1, 6)
2 x**2
```

py

```
array([ 1,  4,  9, 16, 25])
```

```
1 np.sqrt(x)
```

py

```
array([1.          , 1.41421356, 1.73205081,
```

```
1 x + 10
```

py

```
array([11, 12, 13, 14, 15])
```

```
1 x > 3
```

py

```
array([False, False, False,  True,  True])
```

```
1 x % 2 == 0
```

py

```
array([False,  True, False,  True, False])
```

```
1 (x > 1) & (x < 5)
```

py

```
array([False,  True,  True,  True, False])
```

Vectorized conditional choice

`np.where()` is NumPy's equivalent to R's `ifelse()`, both construct a new vector by choosing between two values element-wise based on a condition.

```
1 x = 0:7
2 ifelse(x %% 2 == 0, x, -1)
```

R

```
[1] 0 -1 2 -1 4 -1 6 -1
```

```
1 ifelse(x > 5, "big", "small")
```

R

```
[1] "small" "small" "small" "small" "small" "small" "small"
```

```
1 x = np.arange(8)
2 np.where(x % 2 == 0, x, -1)
```

py

```
array([ 0, -1,  2, -1,  4, -1,  6, -1])
```

```
1 np.where(x > 5, "big", "small")
```

py

```
array(['small', 'small', 'small', 'small', 'small', 'small', 'small', 'small'],
      dtype='<U5')
```

With a single argument, `np.where()` instead returns the positions where the condition is true; the R equivalent is `which()`.

```
1 which(x > 5)
```

R

```
[1] 7 8
```

```
1 np.where(x > 5)
```

py

```
(array([6, 7]),)
```

Subsetting

Indexing conventions

R uses 1-based indexing; Python uses 0-based indexing when subsetting by positive integers.

```
1 x = c("a", "b", "c", "d", "e")
```

R

```
1 x[1]
```

R

```
[1] "a"
```

```
1 x[5]
```

R

```
[1] "e"
```

```
1 x = ["a", "b", "c", "d", "e"]
```

py

```
1 x[0]
```

py

```
'a'
```

```
1 x[4]
```

py

```
'e'
```

In R, a negative index *excludes* a position. In Python, it counts backward from the end (`-1` is the last element).

```
1 x[-1]
```

R

```
[1] "b" "c" "d" "e"
```

```
1 x[-1]
```

py

```
'e'
```

Python slices

A slice has the form `start:stop:step`; `start` is included and `stop` is excluded. Any component may be omitted.

```
1 x = list("abcdefgh"); x
```

py

```
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

```
1 x[1:5]
```

py

```
['b', 'c', 'd', 'e']
```

```
1 x[:4]
```

py

```
['a', 'b', 'c', 'd']
```

```
1 x[4:]
```

py

```
['e', 'f', 'g', 'h']
```

```
1 x[:]
```

py

```
['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']
```

```
1 x[1:7:2]
```

py

```
['b', 'd', 'f']
```

```
1 x[::2]
```

py

```
['a', 'c', 'e', 'g']
```

```
1 x[::-1]
```

py

```
['h', 'g', 'f', 'e', 'd', 'c', 'b', 'a']
```

```
1 x[-3:]
```

py

```
['f', 'g', 'h']
```

Selecting multiple positions

R accepts a vector of positions inside `[ ]`. A base Python list does not: a slice can select a regular range, but arbitrary positions require a comprehension.

```
1 x = letters[1:5]
```

R

```
1 x[c(1, 3, 5)]
```

R

```
[1] "a" "c" "e"
```

```
1 x[c(4, 2, 2)]
```

R

```
[1] "d" "b" "b"
```

```
1 x[c(1, -1)]
```

R

Error in ``x[c(1, -1)]``:
! only 0's may be mixed with negative subscripts

```
1 x = list("abcde")
```

py

```
1 x[[0, 2, 4]]
```

py

TypeError: list indices must be integers or slice

```
1 [x[i] for i in [0, 2, 4]]
```

py

```
['a', 'c', 'e']
```

```
1 [x[i] for i in [3, 1, 1]]
```

py

```
['d', 'b', 'b']
```

```
1 [x[i] for i in [0, -1]]
```

py

```
['a', 'e']
```

Both languages preserve the requested order and repeated positions. R cannot mix positive and negative indexes in one

Integer array indexing

Unlike base Python lists, NumPy arrays accept a list or integer array of positions. As with list indexing, negative positions count backward from the end and can be mixed with positive positions.

```
1 x = np.arange(10, 60, 10); x
```

```
array([10, 20, 30, 40, 50])
```

```
1 x[[0, 2, 4]]
```

```
array([10, 30, 50])
```

```
1 x[[3, 1, 1]]
```

```
array([40, 20, 20])
```

```
1 x[np.array([0, 2, 4])]
```

```
array([10, 30, 50])
```

```
1 x[[0, -1]]
```

```
array([10, 50])
```

Selecting with an integer array or list is called *advanced indexing*; selecting with a scalar or slice is *basic indexing*. The

Out-of-bounds positions

Out-of-bound indexing fails in Python, while R returns a typed missing value for a position that does not exist. Python slices clip each endpoint to the nearest valid boundary.

```
1 x = c(10, 20, 30)
```

R

```
1 x[4]
```

R

```
[1] NA
```

```
1 x[c(1, 4)]
```

R

```
[1] 10 NA
```

```
1 x[-4]
```

R

```
[1] 10 20 30
```

```
1 x = [10, 20, 30]
```

py

```
1 x[4]
```

py

```
IndexError: list index out of range
```

```
1 x[:4]
```

py

```
[10, 20, 30]
```

```
1 x[-4:]
```

py

```
[10, 20, 30]
```

```
1 x[4:]
```

py

```
[]
```

```
1 x[:-4]
```

py

```
[]
```

R - Four additional index types

We have already used positive integers to select positions and negative integers to exclude positions. R's `[]` operator supports four additional index forms:

- logicals - select by `TRUE` positions
- characters - select by names
- zero - select nothing
- empty index - select everything

Logical indexes

Logical subsetting keeps values corresponding to **TRUE**.

Most logical indexes are created by vectorized logical expressions, which return one logical value for each element, often called a *mask*.

```
1 x = c(10, 15, 20, 25, 30)
```

```
1 x %% 20 == 0
```

```
[1] FALSE FALSE TRUE FALSE FALSE
```

```
1 x[x %% 20 == 0]
```

```
[1] 20
```

```
1 (x > 10) & (x < 30)
```

```
[1] FALSE TRUE TRUE TRUE FALSE
```

```
1 x[(x > 10) & (x < 30)]
```

```
[1] 15 20 25
```

```
1 keep = x > 18  
2 keep
```

```
[1] FALSE FALSE TRUE TRUE TRUE
```

```
1 x[keep]
```

```
[1] 20 25 30
```

```
1 x[c(TRUE, NA, FALSE, TRUE, FALSE)]
```

```
[1] 10 NA 25
```

NumPy & Boolean index

NumPy also supports using Boolean arrays for subsetting - positions where the index is **True** are kept and those with **False** are discarded. Boolean ndarrays, lists of Booleans, and the masks produced by vectorized comparisons all work.

```
1 x = np.arange(6)
```

py

```
1 x[np.array([True, False, True, False, True,
```

py

```
array([0, 2, 4])
```

```
1 x[[True, False, True, False, True, False]]
```

py

```
array([0, 2, 4])
```

```
1 x[x % 2 == 0]
```

py

```
array([0, 2, 4])
```

```
1 x[x > 2]
```

py

```
array([3, 4, 5])
```

However, while R recycles logical vectors, NumPy requires a Boolean index to match the dimension it selects.

```
1 x = 0:5
```

R

```
1 x[c(TRUE, FALSE)]
```

R

```
[1] 0 2 4
```

```
1 x[np.array([True, False])] 
```

py

IndexError: boolean index did not match indexed

```
1 x[[True]]
```

py

IndexError: boolean index did not match indexed

Boolean expressions with NumPy

NumPy overloads `&`, `|`, and `~` as element-wise logical operators.

Parenthesize every comparison because these operators have unexpected precedence.

```
1 x = np.arange(10)
```

py

```
1 (x > 2) & (x < 7)
```

py

```
array([False, False, False,  True,  True,
       False])
```

```
1 (x < 2) | (x > 7)
```

py

```
array([ True,  True, False, False, False,
       True])
```

```
1 x[(x > 2) & (x < 7)]
```

py

```
array([3, 4, 5, 6])
```

```
1 x[~(x % 2 == 0)]
```

py

```
array([1, 3, 5, 7, 9])
```

Character indexes

Names allow values to be selected independently of their positions.

```
1 x = c(a = 10, b = 20, c = 30)
```

```
1 x["a"]
```

```
a  
10
```

```
1 x[c("c", "a", "a")]
```

```
 c  a  a  
30 10 10
```

```
1 x["d"]
```

```
<NA>  
NA
```

```
1 x[c("a", "d")]
```

```
 a <NA>  
10  NA
```

```
1 unname(x["a"])
```

```
[1] 10
```

Empty and zero indexes

In R an empty index selects everything, a zero index selects nothing. This differs from Python, where `0` is the first position and `:` is used to select everything.

```
1 x = c(10, 20, 30)
```

R

```
1 x[]
```

R

```
[1] 10 20 30
```

```
1 x[0]
```

R

```
numeric(0)
```

```
1 x[NULL]
```

R

```
numeric(0)
```

```
1 x[c(0, 2, 0)]
```

R

```
[1] 20
```

```
1 x = [10, 20, 30]
```

py

```
1 x[:]
```

py

```
[10, 20, 30]
```

```
1 x[0]
```

py

```
10
```

```
1 x[0:0]
```

py

```
[]
```

In R, zeros may appear alongside positive or negative indexes and are ignored. `NULL` and an empty integer vector also

Subsetting and assignment

Subset syntax can appear on the left side of an assignment in all three cases. A base Python list supports assignment to a single position or a slice, while R and NumPy also allow assignment via logical / Boolean and integer indexes.

```
1 x = c(1, 4, 7, 9)
2 x[2] = 2
3 x[x %% 2 != 0] = -1
4 x
```

[1] -1 2 -1 -1

```
1 x = [1, 4, 7, 9]
2 x[1] = 2
3 x[2:4] = [0, 0, 0]
4 x
```

[1, 2, 0, 0, 0]

```
1 x = np.array([1, 4, 7, 9])
2 x[1] = 2
3 x[x % 2 != 0] = -1
4 x
```

array([-1, 2, -1, -1])

A slice assignment can change a list's length. NumPy arrays have fixed size and type - assignment can replace values, but

NumPy views and copies

Basic slicing usually returns a *view* that shares data with the original array.
Advanced indexing returns a *copy*.

```
1 x = np.arange(6)
2 y = x[1:4]
3 y[0] = 99
4 x
```

py

```
array([ 0, 99,  2,  3,  4,  5])
```

```
1 np.shares_memory(x, y)
```

py

True

```
1 x = np.arange(6)
2 z = x[[1, 2, 3]]
3 z[0] = 99
4 x
```

py

```
array([0, 1, 2, 3, 4, 5])
```

```
1 np.shares_memory(x, z)
```

py

False

Exercise 1

Without running the code, determine the result (or error) of each expression.

```
1 x = c(a = 2, b = 4, c = 6, d = 8)
```

R

```
1 x = np.array([2, 4, 6, 8])
```

py

```
1 x[c(1, 3)]
2 x[-c(1, 3)]
3 x[c(TRUE, FALSE)]
4 x[c("d", "b", "z")]
5 x[x > 3 & x < 8]
```

R

```
1 x[[0, 2]]
2 x[-2:]
3 x[np.array([True, False])]
4 x[x > 3 & x < 8]
5 x[(x > 3) & (x < 8)]
```

py

Matrices & arrays

R matrices and NumPy arrays

Both are homogeneous, multidimensional containers designed for vectorized computation.

```
1 x = matrix(1:6, nrow = 2, ncol = 3) R
2 x
```

```
      [,1] [,2] [,3]
[1,]     1     3     5
[2,]     2     4     6
```

```
1 typeof(x) R
```

```
[1] "integer"
```

```
1 dim(x) R
```

```
[1] 2 3
```

```
1 x = np.array([[1, 2, 3],
2               [4, 5, 6]]) py
3 x
```

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
1 x.dtype py
```

```
dtype('int64')
```

```
1 x.shape py
```

```
(2, 3)
```

Creating arrays

```
1 matrix(0, nrow = 2, ncol = 3)
```

R

```
      [,1] [,2] [,3]  
[1,]    0    0    0  
[2,]    0    0    0
```

```
1 array(1, dim = c(2, 2, 2))
```

R

```
, , 1
```

```
      [,1] [,2]  
[1,]     1     1  
[2,]     1     1
```

```
, , 2
```

```
      [,1] [,2]  
[1,]     1     1  
[2,]     1     1
```

```
1 diag(3)
```

R

```
      [,1] [,2] [,3]  
[1,]     1     0     0  
[2,]     0     1     0  
[3,]     0     0     1
```

```
1 seq(0, 1, length.out = 5)
```

R

```
[1] 0.00 0.25 0.50 0.75 1.00
```

```
1 np.zeros((2, 3))
```

py

```
array([[0., 0., 0.],  
       [0., 0., 0.]])
```

```
1 np.ones((2, 2, 2))
```

py

```
array([[[1., 1.],  
        [1., 1.]],
```

```
       [[1., 1.],  
        [1., 1.]])
```

```
1 np.eye(3)
```

py

```
array([[1., 0., 0.],  
       [0., 1., 0.],  
       [0., 0., 1.]])
```

```
1 np.linspace(0, 1, 5)
```

py

```
array([0. , 0.25, 0.5 , 0.75, 1.  ])
```

Column-major vs row-major

R matrices use *column-major* ordering - a sequence of values fills the matrix down each column. NumPy arrays default to *row-major* ordering - values fill across each row.

```
1 x = matrix(1:6, nrow = 2)
2 x
```

R

```
      [,1] [,2] [,3]
[1,]     1     3     5
[2,]     2     4     6
```

```
1 y = matrix(1:6, nrow = 3, byrow = TRUE)
2 y
```

R

```
      [,1] [,2]
[1,]     1     2
[2,]     3     4
[3,]     5     6
```

```
1 c(y)
```

R

```
[1] 1 3 5 2 4 6
```

```
1 x = np.arange(1, 7).reshape((2, 3))
2 x
```

py

```
array([[1, 2, 3],
       [4, 5, 6]])
```

```
1 y = np.array([[1, 2],
2               [3, 4],
3               [5, 6]])
4 y
```

py

```
array([[1, 2],
       [3, 4],
       [5, 6]])
```

```
1 y.flatten()
```

py

```
array([1, 2, 3, 4, 5, 6])
```

Two-dimensional indexing

Dimensions are separated by commas in both languages; ranges select rectangular regions.

```
1 x = matrix(1:16, nrow = 4, byrow = TRUE); x
```

	[,1]	[,2]	[,3]	[,4]
[1,]	1	2	3	4
[2,]	5	6	7	8
[3,]	9	10	11	12
[4,]	13	14	15	16

```
1 x[1, 2]
```

```
[1] 2
```

```
1 x[2:3, 2:4]
```

	[,1]	[,2]	[,3]
[1,]	6	7	8
[2,]	10	11	12

```
1 x[c(1, 3), c(2, 4)]
```

	[,1]	[,2]
[1,]	2	4
[2,]	10	12

```
1 x = np.arange(1, 17).reshape(4, 4); x
```

```
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12],
       [13, 14, 15, 16]])
```

```
1 x[0, 1]
```

```
np.int64(2)
```

```
1 x[1:3, 1:4]
```

```
array([[ 6,  7,  8],
       [10, 11, 12]])
```

```
1 x[np.ix_([0, 2], [1, 3])]
```

```
array([[ 2,  4],
       [10, 12]])
```

Integer arrays in 2D

An integer list can select along either dimension. When both dimensions receive integer lists, NumPy pairs them element by element instead of crossing them.

```
1 x = np.arange(1, 17).reshape(4, 4)
2 x[[0, 2], :]
```

```
array([[ 1,  2,  3,  4],
       [ 9, 10, 11, 12]])
```

```
1 x[:, [1, 3]]
```

```
array([[ 2,  4],
       [ 6,  8],
       [10, 12],
       [14, 16]])
```

```
1 x[[0, 2], [1, 3]]
```

```
array([ 2, 12])
```

```
1 x[np.ix_([0, 2], [1, 3])]
```

```
array([[ 2,  4],
       [10, 12]])
```

The paired expression selects (row 0, column 1) and (row 2, column 3), not a 2×2 rectangle; `np.ix_()` produces the rectangle.

Reductions and axes

A *reduction* combines many values into fewer values. NumPy's `axis` argument specifies which dimension is collapsed.

```
1 x = matrix(1:12, nrow = 3, byrow = TRUE) R
```

```
      [,1] [,2] [,3] [,4]  
[1,]     1     2     3     4  
[2,]     5     6     7     8  
[3,]     9    10    11    12
```

```
1 sum(x) R
```

```
[1] 78
```

```
1 rowMeans(x) R
```

```
[1] 2.5 6.5 10.5
```

```
1 colMeans(x) R
```

```
[1] 5 6 7 8
```

```
1 x = np.arange(1, 13).reshape(3, 4); py
```

```
array([[ 1,  2,  3,  4],  
       [ 5,  6,  7,  8],  
       [ 9, 10, 11, 12]])
```

```
1 x.sum() py
```

```
np.int64(78)
```

```
1 x.mean(axis=1) py
```

```
array([ 2.5,  6.5, 10.5])
```

```
1 x.mean(axis=0) py
```

```
array([5., 6., 7., 8.])
```

NumPy broadcasting

NumPy broadcasts by shape

R recycles by comparing lengths; NumPy *broadcasts* by comparing shapes.

Shapes are compared from the rightmost dimension to the left, and each pair of dimensions is compatible when they are equal or one is 1.

```
1 x = np.arange(1, 7)
```

py

```
1 x + 10
```

py

```
array([11, 12, 13, 14, 15, 16])
```

```
1 x + np.array([10])
```

py

```
array([11, 12, 13, 14, 15, 16])
```

```
1 x + np.array([10, 100])
```

py

ValueError: operands could not be broadcast toge

```
1 x + np.array([10, 100, 1000,  
2               10_000, 100_000, 1_000_000])
```

py

```
array([ 11, 102, 1003, 10004, 1000
```

Broadcasting across rows

A one-dimensional array aligns with the trailing dimension, so it operates across every row.

```
1 x = np.arange(12).reshape(4, 3)
2 y = np.array([10, 20, 30])
```

py

```
1 x
```

py

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

```
1 x + y
```

py

```
array([[10, 21, 32],
       [13, 24, 35],
       [16, 27, 38],
       [19, 30, 41]])
```

```
1 x.shape
```

py

```
(4, 3)
```

```
1 y
```

py

```
array([10, 20, 30])
```

```
1 y.shape
```

py

```
(3,)
```

```
1 (x + y).shape
```

py

```
(4, 3)
```

Broadcasting down columns

To align with rows, give the vector an explicit singleton column dimension.

```
1 x = np.arange(12).reshape(3, 4)
2 y = np.array([10, 20, 30])
```

py

```
1 y.shape
```

py

```
(3,)
```

```
1 y[:, np.newaxis]
```

py

```
array([[10],
       [20],
       [30]])
```

```
1 y[:, np.newaxis].shape
```

py

```
(3, 1)
```

```
1 x + y[:, np.newaxis]
```

py

```
array([[10, 11, 12, 13],
       [24, 25, 26, 27],
       [38, 39, 40, 41]])
```

Pairwise combinations

Broadcasting two singleton dimensions produces every pairwise combination.

```
1 x = 1:3
2 y = c(10, 20, 30, 40)
3 outer(x, y, FUN = "+")
```

R

	[,1]	[,2]	[,3]	[,4]
[1,]	11	21	31	41
[2,]	12	22	32	42
[3,]	13	23	33	43

```
1 x = np.arange(1, 4)
2 y = np.arange(10, 50, 10)
3 x[:, np.newaxis] + y
```

py

```
array([[11, 21, 31, 41],
       [12, 22, 32, 42],
       [13, 23, 33, 43]])
```

Standardizing columns

Broadcasting makes it possible to transform every column using its own mean and standard deviation.

```
1 rng = np.random.default_rng(523)
2 x = rng.normal(
3     loc=[-1, 0, 1],
4     scale=[1, 2, 3],
5     size=(1000, 3)
6 )
```

```
1 means = x.mean(axis=0)
2 sds = x.std(axis=0)
3 means
```

```
array([-1.01363555,  0.06318758,  0.844760
```

```
1 sds
```

```
array([0.94966875, 1.94205801, 2.8675822 ]
```

```
1 z = (x - means) / sds
2 z.mean(axis=0)
```

```
array([-4.35207426e-17, -7.28306304e-17,
```

```
1 z.std(axis=0)
```

```
array([1., 1., 1.]
```

Unintended broadcasting

Valid shapes do not guarantee the intended calculation.

```
1 x = np.array([[1, 2, 3],
2               [4, 5, 6],
3               [7, 8, 9]])
4 row_means = x.mean(axis=1)
```

py

```
1 x - row_means
```

py

```
array([[ -1.,  -3.,  -5.],
       [  2.,   0.,  -2.],
       [  5.,   3.,   1.]])
```

```
1 x - row_means[:, np.newaxis]
```

py

```
array([[ -1.,   0.,   1.],
       [ -1.,   0.,   1.],
       [ -1.,   0.,   1.]])
```

Both run, but only the second subtracts each row's own mean.

When an expression is surprising, inspect `.shape` before the values. `np.broadcast_shapes()` can predict the result shape

Exercise 2

For each pair of NumPy shapes, determine whether broadcasting succeeds and, if so, the result shape.

1. $(128, 128, 3)$ and $(3,)$
2. $(8, 1, 6, 1)$ and $(7, 1, 5)$
3. $(2, 1)$ and $(8, 4, 3)$
4. $(3, 1)$ and $(15, 3, 5)$
5. $(3,)$ and $(4,)$

Then write a vectorized NumPy expression that subtracts the mean of each row from a matrix x with shape $(100, 5)$.

Comparing R & Python

Vectorization summary

Concept	R	Python / NumPy
element-wise arithmetic	built into atomic vectors	NumPy arrays
element-wise comparisons	built into atomic vectors	NumPy arrays
logical operators	<code>&, , !</code>	<code>&, , ~</code>
element-wise choice	<code>ifelse()</code>	<code>np.where()</code>
scalar repetition	recycling	broadcasting
unequal sizes	repeat by total length	compare shapes from the right
explicit iteration	<code>for</code> , later <code>lapply()</code> / <code>purrr</code>	comprehension, <code>for</code>
reduction	<code>sum()</code> , <code>mean()</code> , <code>rowMeans()</code>	<code>.sum()</code> , <code>.mean(axis=...)</code>

Subsetting summary

Goal	R	Python / NumPy
first element	<code>x[1]</code>	<code>x[0]</code>
last element	<code>x[length(x)]</code>	<code>x[-1]</code>
exclude first	<code>x[-1]</code>	<code>x[1:]</code>
regular range	<code>x[2:5]</code>	<code>x[1:5]</code>
arbitrary positions	<code>x[c(1, 3)]</code>	<code>x[[0, 2]]</code> (NumPy)
Boolean filter	<code>x[x > 0]</code>	<code>x[x > 0]</code> (NumPy)
all rows, second column	<code>x[, 2]</code>	<code>x[:, 1]</code>
independent copy	usually automatic on modification	<code>x.copy()</code> (NumPy)

Takeaways

- R atomic vectors and NumPy arrays support element-wise operations; Python lists require explicit iteration or comprehensions.
- R indexes from 1 and uses negative indexes for exclusion; Python indexes from 0, counts backward with negative indexes, and uses half-open slices.
- R's `[]` supports integer, logical, and name-based selection. NumPy adds integer-array and Boolean indexing to Python's usual indexing syntax.
- Basic NumPy slices are usually views; advanced indexing is a copy. Make copying intentional when the result will be modified.
- R recycling is based on vector length; NumPy broadcasting is based on compatible shapes. When in doubt, inspect `.shape`.